HACKARUN

// HANDOFF PARA A EQUIPE

GUIA DO PROJETO

Arquitetura, convenções, infraestrutura, inscrições e como contribuir.

Versão 0.1.0 · atualizado em 11 de outubro de 2026
https://github.com/Marcos-Codfy/lp-hackarun

REALIZAÇÃO

UNICATÓLICA

PARCERIA

CAPIM DOURADO SHOPPING

> Documento de handoff: tudo que uma pessoa nova precisa pra rodar, entender, alterar e publicar a landing page. Mantido junto do código. O PDF é gerado a partir deste arquivo com `npm run docs:pdf`.

1. Links rápidos

Item	Valor
Repositório (público)	https://github.com/Marcos-Codfy/lp-hackarun
Branch de produção	main
Hospedagem	Cloudflare Pages, plano gratuito (https://lp-hackarun.pages.dev depois de conectar)
Inscrições	Formulário do site, Google Apps Script (Web App) e Google Sheets
Aviso de privacidade	/privacidade (src/pages/privacidade.astro), linkado no consentimento e no rodapé
Node.js	24 LTS (arquivo .nvmrc), mínimo 22.12
Scripts de QA	scripts/qa/ (Playwright + Lighthouse, instalados à parte; seção 13)
Especificação de design	docs/design/design-spec.md (v2, vigente)
Decisões de arquitetura	docs/adr/ (0001 a 0006)
Stack e padrões (PDF)	docs/pdf/HackaRun_LP_Stack_e_Padroes.pdf

2. O que é

Landing page do **HackaRun 2026**, o hackathon dos cursos de Engenharia de Software e Inteligência Artificial da **Unicatólica**, com parceria do **Capim Dourado Shopping**. Evento de 11 a 14 de novembro de 2026, com minicursos, mentoria de egressos, corrida de missões (os NPCs dão pistas) e premiação no shopping com telão e banda.

Objetivos da página: explicar o evento, mostrar a programação e as missões, listar as equipes confirmadas e **converter em inscrições**. Slogan oficial: **"Desvende. Programe. Conquiste."**

A versão atual é a **v2 cinematográfica** (ADR 0006): identidade oficial, pós-processamento real e um objeto 3D novo para cada seção. A v1 ficou só no histórico do Git.

3. Decisões principais

Decisão	Escolha	Motivo curto
Framework	Astro (site estático) + TypeScript	HTML pronto no build, quase zero JS inicial, ótimo LCP e SEO
3D	Partículas procedurais (Three.js + GLSL), 8 objetos	0 KB de vídeo/modelo, nítido em qualquer tela, muda com o scroll
Visual	Pós-processamento (bloom, lente) + câmera coreografada	Pedido do cliente: impacto de cinema, não "correto demais"

Decisão	Escolha	Motivo curto
Identidade	Logo, paleta e fontes oficiais; til da Orbitron corrigido	Fidelidade ao manual da marca; "NÃO" legível em título grande
Animação	GSAP + ScrollTrigger + SplitText, Lenis	Padrão de mercado pra scroll storytelling, gratuito
Hospedagem	Cloudflare Pages (grátis)	Banda e requisições ilimitadas pra estático, CDN no Brasil
Inscrições	Google Sheets via Apps Script	Gratuito, a organização já usa Google, planilha vira o painel
Arquitetura	Clean Architecture + DDD + MVVM	Camadas claras, regras testáveis, fácil de dividir entre pessoas
QA	Playwright + Brave instalado, Lighthouse (scripts/qa)	Captura visual em várias larguras e métricas reais de GPU

Detalhes e alternativas descartadas estão em `docs/adr/`.

4. Como rodar

```
git clone https://github.com/Marcos-Codfy/lp-hackarun.git
cd lp-hackarun
npm ci
cp .env.example .env
npm run dev
```

Abra `http://localhost:4321`. Sem `PUBLIC_REGISTRATION_ENDPOINT` o formulário roda em **modo simulado** no ambiente de desenvolvimento (não grava em lugar nenhum).

Script	O que faz
npm run dev	Servidor local com hot reload
npm run build	astro check + build de produção em dist/
npm run preview	Serve o dist/ localmente
npm run lint / lint:fix	ESLint (inclui as regras de dependência entre camadas)
npm run format	Prettier em tudo (a CI roda format:check)
npm test / test:watch	Testes unitários (Vitest)
npm run validate	lint + testes + build (o mesmo que a CI roda)
npm run docs:pdf	Gera os PDFs de documentação (e copia pra DOCS_OUT_DIR)

Parâmetros de URL para testar:

- `?quality=low`, `?quality=medium` ou `?quality=high`: força o tier do 3D e trava o ajuste automático por FPS.
- `?now=2026-11-14T16:00:00-03:00`: desloca o relógio do site (troca de marcos do contador, estado da inscrição).

5. Estrutura de pastas

```
src/
  domain/              regras de negócio puras (entities, value-objects, services, repositories)
  application/         casos de uso, DTOs e portas (Clock, CalendarExporter, RegistrationGateway,
                       PreferenceStore, AudioCue, TickSound)
  infrastructure/      adaptadores: JSON, repositórios, Apps Script, áudio, storage, calendário
    content/           >>> conteúdo editável do site (event.json, stations.json, faq.json...)
    audio/             AudioEngine, cue do globo (arquivo ou sintetizado), sprite de digitação
  assets/
    brand/             logo oficial do HackaRun e logos brancas dos parceiros (PNG)
    fonts/             Orbitron com o til redesenhado (gerada por script)
  presentation/
    app/               App.boot, AppConfig, AppContext e AppEvents (contrato do EventBus)
    layouts/           BaseLayout: head, fontes, SEO, aurora de fundo e canvas 3D
    sections/          uma seção por arquivo; subcomponentes em pastas (hero/, missions/...)
    components/        ui/, layout/, hud/, countdown/, registration/, teams/, partners/,
                       npc/, sound/, motion/ (pausar animações), seo/
    controllers/       header, menu mobile, nav spy, trilho do HUD, theme/ (aurora)
    view-models/       ViewModels (MVVM): estado e lógica de tela, sem DOM
    views/             Views (MVVM): ligam DOM e ViewModel
    formatters/        textos e datas da interface
    motion/            core/ (Lenis, ScrollTrigger, SceneDirector, AnchorNavigator) e animators/
                       (entradas, raios)
    experience/        3D, carregado sob demanda:
      shapes/          uma ShapeStrategy por objeto + utilitários de traço e amostragem
      shaders/         core.glsl, shapes/*.glsl (um por objeto), partículas, estrelas, bokeh, lente
      particles/       ParticleField, ParticleMaterial e ColorGrade (cor por cena)
      post/            pipelines de render, bloom, passe cinematográfico e looks por cena
      camera/          enquadramentos, voos entre cenas e câmera na mão
      layers/          estrelas, bokeh e anel de choque
      scene/           SceneMixer, SceneLayout, ChargeController e ShapeSpin
      quality/         tiers, detector de GPU e ajuste por FPS
      input/           ponteiro (parallax e repulsão)
    styles/            tokens.css (design tokens), base.css, deferred-sections.css (render adiado)
                       e motion-pause.css (o que para com "Pausar animações")
  shared/              utilitários sem regra de negócio (Observable, EventBus, Ticker, math, dom)
  composition/         composition roots: server.ts (build) e client.ts (browser)
  pages/               rotas do Astro (index, privacidade, 404, .ics)
apps-script/           código do Google Apps Script (API de inscrições) + passo a passo
public/                servido como está: og-image.jpg, favicon, _headers, robots, audio/
scripts/
  audio/               render_cues.py: sons sintetizados para public/audio
  fonts/               patch_orbitron_tilde.py: til da Orbitron
  docs/                gerador dos PDFs, build_pdf_assets.py e assets/ (fontes TTF e logos)
  qa/                  QA visual e de performance (Playwright + Lighthouse), com package.json próprio
tests/                 testes unitários espelhando as camadas
docs/                  guia, design spec, asset brief, ADRs, manifesto da stack e PDFs gerados
design/references/     referências visuais e o vídeo teaser (não vão pro site)
```

Páginas geradas no build:

Rota	Arquivo	O que é
/	pages/index.astro	A landing page
/privacidade	pages/privacidade.astro	Aviso de privacidade (LGPD) com a versão vigente do aviso
/404	pages/404.astro	Página de erro, no mesmo visual

Rota	Arquivo	O que é
/hackarun-2026.ics	pages/hackarun-2026.ics.ts	Agenda com todas as atividades (botões da programação e inscrição)

6. Arquitetura

6.1 Camadas e regra de dependência

As dependências sempre apontam para dentro: `presentation` -> `application` -> `domain`. `infrastructure` implementa as interfaces (portas) declaradas em `domain`/`application`. Quem junta tudo são os **composition roots** em `src/composition`. O ESLint **bloqueia** imports proibidos (`eslint.config.js`):

Camada	Pode importar	Bloqueado
domain	nada de fora	outras camadas e frameworks (three, gsap, lenis, astro)
application	domain	infrastructure, presentation, composition e frameworks
presentation	application (casos de uso, DTOs e portas)	infrastructure: o adapter concreto é ligado em composition
shared	nada	qualquer camada (é utilitário sem regra de negócio)

Na prática: uma View ou ViewModel que precisa de armazenamento, som ou rede recebe a **porta** (ex.: `PreferenceStore`, `AudioCue`, `RegistrationGateway` em `application/ports`) pelo construtor, e o `client.ts` decide qual implementação entra.

6.2 Fluxo no build (SSG)

1. `src/pages/index.astro` chama `serverContainer().getLandingPage.execute()`.
2. O caso de uso lê os repositórios estáticos (JSON em `infrastructure/content`), que passam pelos **mappers** e viram entidades/aggregate. Dado inválido quebra o build (fail fast).
3. A aplicação devolve **DTOs** serializáveis, e as seções renderizam HTML puro.
4. Um JSON (`script#app-config`) leva pro browser só o necessário: marcos do contador, dados da inscrição e a URL do som do globo.

6.3 Fluxo no browser

`App.boot()` espera o primeiro frame pintado (o hero, que é o LCP, aparece antes de qualquer JS pesado) e monta tudo por uma fila (`app/BootQueue.ts`): um passo curto por tarefa, cedendo a thread entre eles (`scheduler.yield`, ou `setTimeout` 0 onde não existe). Cada passo aparece na trilha Timings do DevTools como `boot:<nome>`, e um passo que quebra não derruba os seguintes. Ordem:

1. `mountMotion`: Lenis (smooth scroll), ScrollTrigger, o **SceneDirector** (acompanha as seções com `data-scene`) e o `AnchorNavigator` (links `#secao`). Se GSAP ou Lenis quebrar, a página segue no scroll nativo (`NativeScroll`).

2. `mountViews`: Views MVVM (contador, som, pausar animações, equipes ao vivo, formulário, HUD, NPCs). Funcionam sem 3D.
3. `mountControllers`: controladores de DOM (header e menu primeiro, nav spy, trilho do HUD, aurora). Não usam GSAP, então um animador quebrado não leva eles junto.
4. `mountAnimators`: o pin das missões e as entradas animadas, que só preparam o setup pesado quando a seção chega perto da tela (`motion/core/ApproachWatcher.ts`); o SplitText é carregado sob demanda (`motion/core/splitText.ts`).
5. Segunda fila, com o layout já no lugar: trilha de cenas, `SceneHoldWatcher` (carga do globo final) e, depois do `load` com o navegador ocioso, o chunk do 3D (`mountExperience`). Sem WebGL2, com `save-data` ou se algo falhar, ele emite `experience:fallback` e o brilho em CSS continua.

6.4 MVVM

- **ViewModel** (`presentation/view-models`): estado em `Observable`, chama casos de uso, não toca no DOM. Testável.
- **View** (`presentation/views`): assina o ViewModel, atualiza o DOM e repassa eventos do usuário.
- **Model**: casos de uso + domínio.

6.5 Controladores de DOM

Em `presentation/controllers`, montados por `mountControllers` (não são MVVM e funcionam sem GSAP): `HeaderScrollController` (header de vidro que reage ao scroll), `MobileMenuController` + `FocusTrap` + `ScrollLock` (overlay do menu no celular), `NavSpyController` (link ativo), `HudRailController` (tiques do trilho do HUD) e `SceneThemeController` (cores da aurora por cena). Os links internos ficam com o `AnchorNavigator` (`motion/core`): rola com o driver de scroll, atualiza o hash sem pulo e leva o foco para a seção.

6.6 Eventos entre módulos

Os módulos conversam pelo `EventBus` tipado (`presentation/app/AppEvents.ts`):

Evento	Quando
scene:enter	Uma seção com data-scene chegou ao centro da tela
experience:ready	O 3D desenhou o primeiro frame (informa o tier)
experience:fallback	O 3D não vai rodar (sem WebGL, save-data, erro de carga ou de shader)
experience:charging	A seção com data-scene-hold entrou: o globo começa a carregar
experience:charged	Fim da carga: dispara flash, som e revela o CTA (emitido mesmo sem WebGL)
sound:changed	O usuário ligou ou desligou o som
motion:paused	O usuário pausou ou retomou as animações (seção 6.7)
ui:type	Um caractere foi "digitado" na tela (terminal do NPC, protocolo)
npc:found	Um token NPC foi encontrado

Evento	Quando
registration:intent	Clique num CTA de inscrição (guarda a origem)
registration:submitted	Inscrição enviada com sucesso (protocolo)

6.7 Pausar animações (WCAG 2.2.2)

Tudo que se mexe sozinho por mais de 5 s (3D, raios do logo, loops em CSS) para com o botão **Pausar animações**: no header ao lado do som (`components/motion/PauseMotionToggle.astro`, ícone com tooltip) e no menu do celular (`PauseMotionSwitch.astro`, com texto). Os dois têm `data-motion-toggle`, então andam juntos.

- **Estado:** `MotionPreferenceViewModel` (`view-models/motion`). Começa rodando e a escolha fica salva no navegador (porta `PreferenceStore`, chave `motion`). Com `prefers-reduced-motion` o sistema já parou tudo: o botão fica travado em "pausado" (`aria-disabled`) e não grava nada.
- **DOM:** `MotionPreferenceView` liga os botões (nome acessível fixo, estado no `aria-pressed`, tooltip com a ação do próximo clique, Esc esconde o tooltip) e põe a classe `is-motion-paused` no `<html>` (`MotionPauseSignal`). O evento `motion:paused` sai a cada troca.
- **Quem obedece:** o CSS lê a classe direto; o `FrameLoop` do 3D congela o relógio (desenha um frame parado e, ao retomar, segue dali sem salto) e os raios do logo param. Os dois leem o `MotionPauseSignal` ao montar, então já nascem no estado certo.
- **Loops em CSS:** `styles/motion-pause.css` tem a **lista explícita** de animações infinitas que recebem `animation-play-state: paused`. Um "pausa tudo" travaria no quadro zero as entradas de uma vez só (hero, menu) e o conteúdo sumiria. **Criou um loop infinito novo em CSS? Inclua a classe nessa lista.** Spinner de envio fica de fora (é status, não enfeite).
- Sem JavaScript os botões continuam `hidden`: não haveria o que pausar além do CSS.

7. Experiência 3D

7.1 Cenas e objetos

Um único `THREE.Points` guarda um alvo de posição por cena; o morph inteiro acontece no vertex shader.

Cena	Seções	Objeto	Arquivos
core	Hero e Inscrição	Globo de energia em fitas, borda acesa por fresnel	CoreShape.ts, core.glsl
orbit	O evento	Cronômetro-giroscópio: mostrador, ponteiro e anéis em cardã	OrbitShape.ts, orbit.glsl
enigma	NPCs	"?" extrudado com faixa tracejada e auréola de HUD	EnigmaShape.ts, enigma.glsl
map	Missões	Mapa em holograma: plataforma, rota em espiral, 8 faróis	MapShape.ts, map.glsl
terrain	Programação e Mentores	Terreno synthwave: grade rolando, montanhas e sol em faixas	TerrainShape.ts, terrain.glsl

Cena	Seções	Objeto	Arquivos
stage	Grande final	Palco: telão de LED, treliça, canhões de luz e plateia	StageShape.ts, stage.glsl
trophy	Prêmios	Troféu de torno em linhas, alças e estrela	TrophyShape.ts, trophy.glsl
dust	Equipes e FAQ	Galáxia distante (bojo, dois braços, estrelas soltas)	DustShape.ts, dust.glsl

Os `.ts` ficam em `experience/shapes/` e os `.glsl` em `experience/shaders/shapes/` (o globo em `experience/shaders/core.glsl`). Funções comuns das formas estão em `shaders/shapes/helpers.glsl`.

7.2 Como o morph funciona

1. Cada seção tem `data-scene="<chave>"`. O `SceneDirector` transforma o scroll num índice contínuo (`scenes.progress`, ex.: 3,4 mistura a cena 3 e a 4), mais a velocidade do scroll.
2. O `SceneMixer` converte o índice no trecho atual (forma de origem, de destino e progresso suavizado).
3. Cada `ShapeStrategy` gera a nuvem de pontos estática da forma (uma vez, com semente fixa). A geração roda num **Web Worker** (`shapes/shapes.worker.ts`, chamado pelo `ShapeSource`), que devolve os buffers por transferência enquanto a GPU compila os shaders; sem worker, o mesmo cálculo roda na thread principal em fatias (`ParticleJob`). Todas as formas usam a mesma permutação, então a partícula "i" ocupa o mesmo lugar relativo em toda forma e a troca não vira chuvisco.
4. No vertex shader, cada forma tem `shape<Nome>(p)` (deformação viva: giro, ondas, pulsos) e `<nome>Heat(p)` (calor que puxa a cor pra crista ou pro acento). A troca é uma onda que varre a tela, e no meio do voo as partículas escorrem por um fluxo contínuo.
5. O `ParticleMaterial` monta o shader com `defines`: cena ausente na página nem declara o atributo (`HAS_<CENA>`), e o compilador remove o ramo inteiro.

7.3 Pipeline de pós-processamento

Em `experience/post/`, escolhido pelo nível de efeitos do tier:

- `ComposerPipeline` (tiers médio e alto): `RenderPass` -> `ScaledBloomPass` (UnrealBloomPass com resolução própria e tinta por cena) -> `CinematicPass` (`shaders/cinematic.frag.glsl`: onda de choque, zoom blur da travessia, aberração cromática, exposição com ombro suave, flash, vinheta e grão). Buffers HalfFloat guardam o excesso de luz.
- `DirectPipeline` (tier baixo): render direto; o sprite da partícula carrega um halo "fake" e o anel de choque vem da malha 3D.
- `PostDirector` mistura os `SCENE_LOOKS` (bloom, raio, limiar, tinta, exposição, estrelas, bokeh) com os mesmos pesos do morph e soma os eventos (carga, flash, travessia). O `StageRenderer` troca de pipeline em tempo de execução.

7.4 Câmera, camadas e cor

- **Câmera** (`camera/`): `CameraShots` define o enquadramento de cada cena (truck, crane, dolly, mira, FOV, órbita), `FlightPath` a coreografia de cada travessia (ex.: atravessar o globo rumo à próxima cena) e `Handheld` a deriva de "câmera na mão" e o tremor da carga. `PointerTracker` dá parallax e repulsão no desktop.
- **Camadas** (`layers/`): `StarField` (céu ao fundo), `BokehLayer` (luzes desfocadas na frente) e `ShockwaveRing` (anel do clímax), orquestradas pela `Atmosphere`.
- **Cor**: `particles/ColorGrade.ts` dá a cada forma fundo, corpo, crista e acento da paleta oficial. Atrás do canvas, a **aurora** (`BaseLayout`) muda de cor por cena via `SceneThemeController` (`controllers/theme/sceneThemes.ts`); as cores são `@property <color>`, então a troca interpola. O canvas é opaco e soma sobre a aurora em `mix-blend-mode: screen`.

7.5 Clímax da inscrição

A seção `#inscricao` tem `data-scene="core"` e `data-scene-hold="1500"`. Quando ela entra, o `SceneHoldWatcher` emite `experience:charging` e conta 1,5 s **em tempo** (o scroll nunca é travado). No fim emite `experience:charged`: o `ChargeController` estoura o globo (flash, pulso, anel e brilho residual), o `SoundBusBinder` toca o cue (uma vez por carga, via `ChargeCueGate`) e o CTA aparece. Sem WebGL o evento sai do mesmo jeito.

7.6 Tiers de qualidade

Tier	Partículas	DPR máx.	Pós	Estrelas	Bokeh	Quem recebe
low	9.000	1	sem bloom (halo fake)	900	12	GPU por software, celular sem GPU forte
medium	22.000	1,5	bloom em meia resolução	1.800	22	celular capaz, notebook com GPU integrada
high	42.000	2	bloom inteiro	3.200	30	GPU dedicada ou Apple M; desktop com 8+ núcleos

- O tier inicial vem do `QualityDetector` (nome da GPU, núcleos, memória, tela e tipo de ponteiro).
- O `AdaptiveQuality` mede o FPS: abaixo de 45 por ~2 s corta um degrau do pós (inteiro -> meia -> nenhum), depois o DPR, depois o tier. Nunca sobe de volta.
- O render pausa com a aba oculta e congela com "Pausar animações" (seção 6.7). Com `prefers-reduced-motion`, a cena troca inteira e o loop desenha sob demanda.

7.7 Como adicionar uma cena

1. Adicione a chave em `SceneKey` (`presentation/app/AppEvents.ts`). O TypeScript passa a apontar todos os `Record<SceneKey, ...>` que faltam: `ColorGrade.ts`, `SceneLooks.ts`, `CameraShots.ts`, `SceneLayout.ts`, `ParticleField.ts` (nome do atributo) e `sceneThemes.ts` (aurora).

2. Crie a `ShapeStrategy` em `experience/shapes/<Nome>Shape.ts` (`generate(count, random)` devolve xyz em sequência, dentro de `SHAPE_RADIUS`) e registre em `ShapeFactory.createDefault()`. Prefira os utilitários `strokes.ts`, `PointBrush` e `PolylinePath` (traços finos e contínuos em qualquer tier).
3. Crie `experience/shaders/shapes/<chave>.glsl` com `vec3 shape<Nome>(vec3 p)` e `float <chave>Heat(vec3 p)`. Constantes compartilhadas com o `.ts` ficam espelhadas nos dois arquivos, com comentário avisando.
4. Declare o atributo em `shaders/common.glsl` (`#ifdef HAS_<CHAVE>` / `attribute vec3 a<Nome>;`), adicione o bloco `ADD_SHAPE` em `particles.vert.glsl` e inclua o `.glsl` e a chave (`ALL_KEYS`) no `ParticleMaterial.ts`.
5. (Opcional) Uma curva de travessia em `camera/FlightPath.ts`.
6. Na seção: `data-scene="<chave>"` e `data-checkpoint="<rótulo do HUD>"`.
7. Confira com `?quality=low` e `?quality=high`, no celular e no desktop (seção 13).

8. Editar conteúdo sem programar

Tudo em `src/infrastructure/content/`. Edite, rode `npm test` (os testes validam o conteúdo real) e abra um PR.

Arquivo	O que controla
event.json	Datas, locais, marcos do contador e inscrição (endpoint, closesAt, capacity, teamSize)
stations.json	As 8 missões e a pontuação (a soma dos critérios precisa dar 100)
speakers.json	Instrutores, mentores, palestra e banca (name: null = card "a revelar")
prizes.json	Prêmios (reward: null = "a revelar")
partners.json	Realização e parceiros (nome, papel e link)
faq.json	Perguntas frequentes
teams.json	Lista reserva de equipes (as oficiais vêm da planilha, ao vivo)

Textos fixos de seção (títulos, chamadas) ficam nos próprios `.astro` em `presentation/sections/`.

9. Inscrições (Google Sheets)

Fluxo: formulário -> validação no navegador (regras do domínio) -> `POST` para o Web App do Apps Script -> nova linha na aba **Inscricoes** com protocolo `HR26-XXXXXX`. Para a equipe aparecer no site, a organização muda a coluna **Status** para `confirmado`. O site lê `?action=teams`, que expõe **só** nome da equipe, curso e quantidade de pessoas.

Passo a passo completo em `apps-script/README.md`. Depois de publicar o Web App, coloque a URL em `PUBLIC_REGISTRATION_ENDPOINT` (variável de ambiente na Cloudflare) ou em `registration.endpoint` no `event.json`.

O Apps Script tem três arquivos: `Code.gs` (rotas `doPost`/`doGet`, planilha e protocolo), `Validation.gs` (regras do servidor) e `Limits.gs` (prazo, capacidade e limite de envios). Os testes

rodam os três de verdade no Node, com dublês dos serviços do Google: `npx vitest run tests/infrastructure/apps-script`.

9.1 Prazo e capacidade em tempo de execução

O build fecha o formulário pelo `event.json` (`closesAt`, `capacity`), mas qualquer um pode chamar a URL do Web App direto. Por isso o `Limits.gs` confere **a cada envio** as Propriedades do script e responde `closed` quando não dá mais:

Propriedade	O que faz
CLOSES_AT	Prazo final, data ISO com hora e fuso (2026-11-08T23:59:00-03:00)
CAPACITY	Máximo de equipes: conta as linhas Equipe que não estão recusado
INSCRICOES_FECHADAS	sim fecha na hora, sem novo deploy do site

- Lotou, fecha pra todo mundo (equipe e individual), igual ao site. Recusar uma equipe na coluna Status libera a vaga.
- Valor inválido (`CAPACITY=trinta`) **fecha** as inscrições e grava o erro no log: melhor errar pro lado seguro.
- `URL/exec?action=health` responde `open: true/false` sem ler a planilha: serve pra conferir a configuração.
- Mantenha `CLOSES_AT` e `CAPACITY` iguais a `registration.closesAt` e `registration.capacity` do `event.json`.

9.2 Limites de envio

- **Por e-mail:** até 10 tentativas a cada 10 min por e-mail do líder (guardado como hash no `CacheService`, nunca em texto). Cobre os reenvios automáticos do site e segura quem fica testando se um e-mail já está inscrito.
- **Global:** 120 envios por minuto. O e-mail esbarra no próprio limite antes de contar no global, então um script repetindo o mesmo e-mail não tranca a fila de todo mundo.
- Resposta `rate-limited`: o `RetryingRegistrationGateway` tenta de novo com espera crescente (até 3 tentativas, só para `network`, `server` e `rate-limited`).

9.3 Consentimento (LGPD)

- O checkbox do formulário linka o aviso em `/privacidade` (abre em nova aba), que também está no rodapé.
- O envio leva `consentVersion` (a constante `CONSENT_VERSION` de `application/use-cases/SubmitRegistration.ts`) e `consentAt`. O servidor exige os dois e grava na coluna **Consentimento**: `versão · data/hora ISO do aceite`. Sem isso a inscrição não entra.
- **Mudou o texto do aviso? Suba o `CONSENT_VERSION`.** A página `/privacidade` mostra a versão vigente.

9.4 Idempotência (reenvio sem linha duplicada)

- Cada sessão do formulário gera uma **chave de envio** (UUID, `view-models/registration/idempotencyKey.ts`) que vale para todo reenvio até a inscrição entrar (`SubmissionSession`).
- No servidor, mesma chave + mesmo e-mail devolve o **mesmo protocolo**: primeiro pelo cache (10 min) e, depois disso, pela coluna **Chave de envio** gravada na linha. Mesmo e-mail com outra chave é `duplicate`.
- Planilha criada antes da coluna nova: o script completa o título dela na primeira gravação.

9.5 Outras proteções

Honeypot e tempo mínimo de preenchimento (3 s: `MIN_FILL_MS` no `SubmitRegistration.ts` e `MIN_ELAPSED_MS` no `Validation.gs`, um teste confere), validação completa no servidor (nome, e-mail, WhatsApp, curso, tamanho da equipe, e-mails repetidos), trava de concorrência (`LockService`), proteção contra fórmula injetada na planilha (`=`, `+`, `-`, `@`) e cache de 60 s na lista de equipes.

10. Identidade e assets

10.1 Logos e imagens

- **Logo HackaRun:** `src/assets/brand/hackarun-logo.png`, a imagem oficial. Nunca redesenhe; os raios em volta são desenhados por código (`motion/animators/hero/lightning/`) num canvas 2D, só com o hero visível.
- **Parceiros:** `unicatolica-white.png` e `capim-dourado-white.png` (versões brancas oficiais) aparecem discretas no hero e grandes na faixa "Quem faz acontecer" antes do rodapé (`components/partners/`).
- **Compartilhamento:** `public/og-image.jpg` (1200×630), usada no Open Graph e no Twitter Card.
- Imagens em `src/assets/` são otimizadas pelo Astro no build. Pedidos de arte em `docs/design/asset-brief.md`.

10.2 Paleta e tipografia

A paleta oficial está em `src/presentation/styles/tokens.css` (detalhes na design spec). Fontes self-hosted, só o subset latin, declaradas no `BaseLayout.astro`:

- **Orbitron** (títulos, botões, números) em `src/assets/fonts/orbitron-pt-latin-wght.woff2`. A original tem um til que, em título grande, parece acento grave ("NÀO"). O script `python scripts/fonts/patch_orbitron_tilde.py` redesenha o til em degraus (Ã Õ ã õ Ñ ñ) a partir da fonte do Fontsource. Requer `fonttools` e `brotli` (`pip install fonttools brotli`) e o `node_modules` instalado.
- **Montserrat** (texto) e **JetBrains Mono** (HUD) vêm direto dos pacotes `@fontsource-variable/*`.

10.3 Sons

Gerados por síntese, sem amostras de terceiros, com `python scripts/audio/render_cues.py` (requer `numpy` e `lameenc`: `pip install numpy lameenc`). Saída em `public/audio/`:

- `cta-globe.mp3`: flash elétrico + grave ("low boom") do clímax da inscrição. Se o arquivo faltar, toca uma versão sintetizada na hora (`SynthAudioCue`). Limite: 60 KB.
- `ui-typing.wav`: sprite com 6 cliques de tecla de 0,09 s (22.050 Hz, mono), tocados a cada evento `ui:type` com variação de tom e intervalo mínimo de 35 ms.

O som começa **desligado** (autoplay e WCAG 1.4.2). O botão de som do header (ícone de equalizador) liga o áudio, e só então os arquivos são baixados. A preferência fica salva no navegador.

10.4 Assets dos PDFs

O pdfkit não lê woff2 variável. `python scripts/docs/build_pdf_assets.py` (requer `fonttools`, `brotli` e `Pillow`) gera instâncias estáticas das fontes da marca e versões menores dos logos em `scripts/docs/assets/`. Sem esses arquivos, os PDFs caem nas fontes padrão (Helvetica/Courier).

11. Convenções de código

- Código, nomes de arquivos e commits em **inglês**. Textos da interface e comentários em **português**.
- Comentário curto e objetivo, só quando o código não se explica sozinho. Nada de comentar o óbvio.
- POO: classes pequenas com uma responsabilidade. **Alvo de até 150 linhas por arquivo, teto de 250**: passou, quebre. O ESLint conta só linhas de código (sem branco nem comentário): **avisa acima de 200 e dá erro acima de 250**, inclusive nos `.astro` (o `<style>` conta). Componente grande vira subcomponentes na pasta da seção (ex.: `sections/npc/NpcTerminal.astro` com `TerminalBar` e `TerminalScreen`).
- Camadas: respeite a tabela da seção 6.1. A apresentação recebe **portas** da aplicação, nunca importa `@infrastructure/*` (o ESLint bloqueia); quem escolhe o adapter é o `src/composition/client.ts`.
- Classes e componentes em `PascalCase`, funções e variáveis em `camelCase`, constantes globais em `UPPER_SNAKE_CASE`.
- Imports por alias de camada: `@domain/*`, `@application/*`, `@infrastructure/*`, `@presentation/*`, `@shared/*`, `@composition/*`.
- `import type` para tipos (obrigatório pelo `verbatimModuleSyntax`). TypeScript no modo mais estrito (`noUncheckedIndexedAccess`, `exactOptionalPropertyTypes`), sem `any`.
- CSS: use **só** os tokens de `styles/tokens.css` (cor da paleta é `var(--c-*)`, texto miúdo claro é `--ink-*`, campos usam `--control-*`); estilos com escopo dentro do `.astro`; media queries `min-width` (mobile first: o estilo base é o do celular e as larguras maiores somam por cima).
- Foco: nunca `outline: none`. Para esconder o contorno de um foco programático use `outline: 2px solid transparent`, que continua visível no modo de alto contraste (cores forçadas).

- Loop de animação infinito em CSS entra na lista do `styles/motion-pause.css` (seção 6.7).
- GLSL: um arquivo por objeto; constante espelhada com o `.ts` tem comentário dos dois lados.
- Acessibilidade e performance não são opcionais (checklist na seção 17).

12. Performance

12.1 Orçamento

Métrica	Meta
Lighthouse mobile	Performance >= 90 · Acessibilidade, Boas práticas e SEO 100
LCP (4G, celular médio)	< 2,5 s
CLS	< 0,05
INP	< 200 ms
JS inicial (gzip)	< 70 KB (three.js fica num chunk lazy)
Fontes	3 woff2 latin (~90 KB no total), 2 com preload
Áudio	Cue até 60 KB; nada é baixado antes de ligar o som
Partículas	low 9k · medium 22k · high 42k

Situação em 10/10/2026 (Lighthouse mobile): acessibilidade, boas práticas e SEO em **100**; performance em otimização.

12.2 Técnicas aplicadas

- **HTML pronto no build:** o H1 do hero é o logo oficial e é o LCP: imagem otimizada pelo Astro, carregada sem lazy e com `fetchpriority` alta; CSS pequeno entra inline.
- **Boot fatiado depois do paint:** `App.boot()` espera o primeiro frame e roda a `BootQueue`, um passo curto por tarefa (seção 6.3): a primeira pintura não espera o JavaScript e o input não fica preso atrás do boot.
- **Setup sob demanda:** animações de entrada montam quando a seção se aproxima da tela (`ApproachWatcher`); o SplitText fica fora do bundle inicial.
- **3D tardio e fora da thread principal:** o chunk do three.js só é importado depois do `load` e com o navegador ocioso. As formas são geradas num Web Worker enquanto os shaders compilam; o `StageCompiler` prepara um programa por tarefa com `KHR_parallel_shader_compile` (`compileAsync`), já no alvo de render certo, antes do primeiro frame.
- **Shader enxuto:** `defines` por cena removem ramos de formas ausentes; índices da paleta são constantes.
- **Render adiado do layout:** `content-visibility: auto` nas seções longe do topo, em `styles/deferred-sections.css` (importado pelo `base.css`, na layer `base`), com `contain-intrinsic-size` (altura do conteúdo, sem o padding da seção) medido em 390 e 1440 px. Mudou bastante o conteúdo de uma dessas seções? Meça de novo e atualize o valor. O `LayoutRefresher` recalcula os gatilhos quando a seção monta, e o scroll por âncora confere

o destino de novo ao chegar (`scripts/qa/anchorcheck.mjs` testa isso).

- **Aurora barata:** manchas em `radial-gradient` sem `filter: blur()`; cores interpoladas por `@property`.
- **Qualidade adaptativa** (seção 7.6), DPR limitado por tier, pausa com a aba oculta, raios do logo só com o hero visível e versão leve em telas de toque.
- **Fallbacks:** sem WebGL2 ou com `save-data`, o 3D não inicia e fica o brilho em CSS.
- **Cache:** `/_astro/*` imutável (nome com hash), áudio por 7 dias (`public/_headers`).

13. QA

Os scripts ficam em `scripts/qa/`, com `package.json` próprio: Playwright e Lighthouse não entram no `npm ci` do projeto, só quem faz QA instala. Guia completo de cada script em `scripts/qa/README.md`.

```
cd scripts/qa
npm install
npx playwright install chromium   # só se a máquina não tiver Brave nem Chrome
```

O navegador sai de `BRAVE_PATH`, depois `CHROME_PATH`, depois do caminho padrão do Brave ou do Chrome no sistema e, por último, do Chromium do Playwright. O navegador instalado roda com **GPU real** (`--enable-gpu`, `--ignore-gpu-blocklist` e `--use-angle=d3d11` no Windows): sem isso o headless renderiza por software e o 3D sempre cai no tier `low`. Rode contra o build de produção (`npm run build && npm run preview`), passando a URL como primeiro argumento; as saídas vão para `scripts/qa/out/` (fora do Git).

13.1 Visual

Script	Uso
shots.mjs	node shots.mjs <url> 390 844 out/m390 "#sobre,#missoes@0.5" --mobile
anchorcheck.mjs	node anchorcheck.mjs <url>: links do header e CTAs param no alvo
overflow.mjs	node overflow.mjs <url> --width=360 --width=390: nada passa da tela

- Capturas por seção em 360/390, 768, 1024, 1366/1440 e 1920 px, com emulação de celular (`--mobile`: `isMobile`, toque, DPR 2) nas larguras pequenas.
- O `shots.mjs` espera a classe `is-experience-ready` no `<html>` (o 3D desenhou) e lista os erros do console no fim.
- Rode também com `?quality=low` e `?quality=high` na URL, com "reduzir movimento" ligado, com o som ligado e com "Pausar animações" ligado.
- Âncoras: o `anchorcheck.mjs` clica nos links e confere se o destino parou na folga do header (`scroll-padding-top`); as seções com render adiado não podem errar o alvo. `anchorcheck` e `overflow` saem com código 1 quando falham.

13.2 Performance

Script	Uso
vitals.mjs	node vitals.mjs <url> --mobile: LCP, CLS, load, tempo até o 3D, em 4G e sem limite
longtasks.mjs	node longtasks.mjs <url> --runs=3: tarefas longas com CPU 4x e 4G lento, com mediana
trace.mjs	node trace.mjs <url>: trace até o 1º paint, downloads e passos boot:*
lighthouse.mjs	node lighthouse.mjs <url> --runs=3: Lighthouse mobile (--desktop) com mediana
lighthouse-summary.mjs	node lighthouse-summary.mjs out/lighthouse/*.json: resume relatórios já gravados

- Conte e atribua as tarefas longas (trace do Chrome) em vez de olhar só o tempo absoluto.
- As medições oscilam: rode várias vezes e compare medianas (os scripts de performance já imprimem a mediana).
- Script novo usa `lib/browser.mjs` e `lib/cli.mjs` (navegador, perfis de rede, argumentos e pasta de saída) e nunca caminho fixo da máquina de alguém.

14. Commits (Conventional Commits)

Formato: `<tipo>(<escopo>): <descrição curta no imperativo, em inglês>`

Tipo	Quando usar
feat	Nova funcionalidade
fix	Correção de bug
perf	Melhoria de performance
refactor	Mudança de código sem alterar comportamento
style	Formatação (sem mudar lógica)
test	Testes
docs	Documentação
build	Build e dependências
ci	Pipeline de CI
chore	Tarefas gerais
revert	Reverter commit

Escopos: `domain`, `application`, `infra`, `ui`, `sections`, `experience`, `motion`, `a11y`, `perf`, `seo`, `content`, `docs`, `ci`, `deps`, `config`, `tests`.

```
feat(sections): add prizes section with locked rewards
fix(experience): stop render loop when tab is hidden
content(faq): add question about team size        <- inválido: "content" é escopo, não tipo
docs(content): update schedule for day 14          <- correto
feat(domain)!: require team size rule              <- "!" = breaking change
```

O **commitlint** valida a mensagem no `commit-msg` e o **lint-staged** roda ESLint e Prettier no `pre-commit` (Husky). Mudanças em `package.json`, `docs/stack-manifest.json` ou neste guia regeneram os PDFs automaticamente.

15. Branches e Pull Requests

1. Atualize a `main` e crie a branch: `feat/<assunto>`, `fix/<assunto>`, `docs/<assunto>`.
2. Commits pequenos, no padrão acima.
3. `npm run validate` antes de abrir o PR.
4. Abra o PR pra `main` usando o template (checklist). A CI roda lint, formatação, testes e build; a Cloudflare gera uma URL de preview.
5. Pelo menos 1 revisão. Merge com **squash**, com título no padrão Conventional Commits.

Recomendado: proteger a `main` no GitHub (Settings -> Branches), exigindo PR e CI verde.

16. Deploy na Cloudflare Pages (grátis)

1. Crie uma conta gratuita em `dash.cloudflare.com`.
2. **Workers & Pages -> Create -> Pages -> Connect to Git** e escolha `Marcos-Codfy/lp-hackarun`.
3. Framework preset **Astro**. Build command `npm run build`. Output directory `dist`.
4. Variáveis de ambiente: `NODE_VERSION=24`, `PUBLIC_SITE_URL=https://<seu-projeto>.pages.dev`, `PUBLIC_REGISTRATION_ENDPOINT=<url do Web App>`.
5. **Save and Deploy**. Cada push na `main` publica em produção; cada PR ganha uma URL de preview.
6. (Opcional) Domínio próprio em **Custom domains**.

Cache e headers de segurança ficam em `public/_headers`.

Opção	Por que não foi a escolhida
Vercel (Hobby)	Termos do plano gratuito proíbem uso comercial/patrocinado; banda limitada a 100 GB/mês
GitHub Pages	Banda sugerida de 100 GB/mês, sem headers customizados e sem preview por PR
Cloudflare Pages	Escolhida: banda e requisições ilimitadas pra estático, CDN com servidores no Brasil, preview por PR

17. Checklist antes do merge

- [ ] `npm run validate` passou (e `npm run format:check`).
- [ ] Testei no celular (360 e 390 px) e no desktop (1440 px), com capturas das seções alteradas (`scripts/qa/shots.mjs`), sem scroll horizontal (`overflow.mjs`) e com as âncoras no alvo (`anchorcheck.mjs`).
- [ ] Mexeu no 3D: conferi com `?quality=low` e `?quality=high`, sem erro no console.
- [ ] Navegação só por teclado funciona; foco visível.
- [ ] Ativei "reduzir movimento" no sistema e a página continua utilizável.
- [ ] "Pausar animações" para tudo o que se mexe sozinho (loop novo em CSS entrou no `motion-pause.css`).
- [ ] Sem texto com contraste baixo; imagens com `alt`.
- [ ] Nenhum arquivo passou de 250 linhas (o ESLint avisa acima de 200); nenhum `console.log` esquecido.
- [ ] Lighthouse mobile sem regressão (acessibilidade, boas práticas e SEO em 100).

18. Versionamento

SemVer em `package.json` (`MAJOR.MINOR.PATCH`). Tags `vX.Y.Z` nos marcos (ex.: `v1.0.0` no lançamento das inscrições). As mensagens no padrão Conventional Commits permitem gerar changelog depois.

19. Pendências

19.1 Da organização

- [ ] URL do Web App de inscrições (`PUBLIC_REGISTRATION_ENDPOINT`).
- [ ] Prazo das inscrições (`registration.closesAt`) e capacidade de equipes (`registration.capacity`).
- [ ] Tamanho mínimo e máximo da equipe (`registration.teamSize`, hoje 2 a 5 provisório; igual ao `TEAM_SIZE` do `apps-script/Validation.gs`).
- [ ] Prêmios (`prizes.json`, os 3 estão "a revelar") e link do regulamento oficial.
- [ ] Nomes de instrutores, mentores, palestrante e banca (`speakers.json`, todos "a revelar").
- [ ] Site oficial do Capim Dourado para o link da faixa de parceiros (`partners.json`, hoje `null`).
- [ ] Local da caça aos NPCs (universidade ou shopping).
- [ ] Aprovar os sons sintetizados (ou enviar os oficiais, até 60 KB).
- [ ] Logos em SVG (opcional; as versões PNG oficiais já estão no site).
- [ ] Domínio próprio (opcional).

19.2 Técnicas

- [ ] Lighthouse mobile >= 90 em performance (em andamento).
- [x] Versionar os scripts de QA (capturas, vitals, tarefas longas, trace, âncoras, overflow e Lighthouse) em `scripts/qa/`.
- [x] Remover dependências da v1 que não são mais usadas (`@fontsource-variable/big-shoulders` e `inter`).

20. Documentação gerada

`npm run docs:pdf` gera em `docs/pdf/`:

- `HackaRun_LP_Stack_e_Padroes.pdf`: stack, bibliotecas (lidas do `package.json`), recursos, padrões e conceitos (só os nomes, vindos de `docs/stack-manifest.json`).
- `HackaRun_LP_Guia_do_Projeto.pdf`: este guia.

Os PDFs usam as fontes da marca (Orbitron, Montserrat e JetBrains Mono, em `scripts/docs/assets/`). Caracteres que essas fontes não têm (setas, sinais de comparação, desenho de caixa) viram equivalentes em ASCII. Se `DOCS_OUT_DIR` estiver definido no `.env`, os PDFs e este guia também são copiados pra essa pasta.